

# 课题管理 Agent Skill 设计方案

https://gemini.google.com/app/f112889180885ac5

**User prompt:** 我现在要编写一个课题skill，首先需要告诉课题名称，课题方向，专业信息，导师信息，引用本课题项目已经有的本地资料或云端资料，然后调用爬虫skill，查看百度文库，维基百科，谷歌学术等信息网站，以及自己专业的相关网站。然后搜其资料后，期间可以将趴下来的图表缓存本地，使用图表内容等数据形成任务书（task.md），然后根据任务书去构建代码，项目报告等具体任务，不同的具体任务要有不同的任务书，方便用来让agent来执行不同的模块。当然后期修改不同的模块和需求时，也应当修改对应的模块任务书。最后所有子任务完成后，对应生成实验方法，实验流程等任务书，我们还需要根据已有的代码，训练数据、结果等，可能是大量的本地图表，可视化数据，讲本地文件用来生成实验报告，给导师汇报的演讲报告，以及使用PPTmaster技能生成ppt，可以即如自己的PPT模板，以及本地素材，都是真实展现于报告和ppt中。同时期间可嫩会使用很多的第三方技能。这里的技能依赖也应有说明。

**Response:** 这个工作流的规划非常清晰且符合复杂工程/科研项目的规范！通过“**任务书驱动（Task-Driven Architecture）**”的模式，能够将复杂的科研/项目开发拆解为可被 Agent 独立、精准执行的子任务，极大地降低大模型在长文本或多任务上下文下的漂移问题。

为了让你能够顺利将这个流程落地为可执行的 Skill（如 Claude Skill / AutoGen / CrewAI / Cursor / Custom Agent 指令集），我为你梳理了一份 **课题管理 Agent Skill 设计方案架构**，你可以直接参考并整理为自己的 Skill 指令：

## 🔧 课题 Skill 架构设计框架

### 1. 基础配置与上下文初始化 (Initialization)

在 Skill 启动时，强制 Agent 收集并锁定的基本信息：

- **课题名称：**例如 基于无电容断电保护的间歇性计算实时系统研究
- **课题方向：**如 嵌入式系统 / 实时操作系统 / 硬件协同设计
- **专业与导师信息：**专业名称、导师学术偏好（如：偏好仿真验证还是实物测试、PPT样式偏好等）
- **上下文数据挂载：**
  - ./data/local/（本地已有的文献、代码、实验数据）
  - ./data/cloud/（云端链接、网盘或知识库索引）

### 2. 信息检索与数据沉淀 (Web Crawler & Retrieval Skill)

调用网络爬虫与学术检索 Skill 进行自动化拓展：

1. **目标平台：**谷歌学术 (Google Scholar)、维基百科 (Wikipedia)、百度文库、IEEE Xplore、ACM Digital Library 及专业论坛/数据库。
2. **数据提取与缓存规则：**
  - **文本/论文：**整理为结构化 Markdown（摘要、关键创新点、对比实验架构）。
  - **图表提取：**将爬取到的架构图、对比图缓存至本地 ./assets/crawled\_images/，并记录元数据（图片来源、Caption、上下文说明）。

### 3. 任务书驱动架构 (Task-Driven Architecture)

核心逻辑：**不直接让 Agent 一口气写完代码或报告，而是先生成对应模块的 task.md。**

```
📁 project-root/
├── 📄 master_task.md          # 总课题规划任务书
├── 📁 tasks/                  # 模块任务书目录
│   ├── 01_literature_review.md # 文献综述与立项依据任务书
│   ├── 02_system_architecture.md # 系统/代码架构设计任务书
│   ├── 03_code_implementation.md # 代码开发与单元测试任务书
│   ├── 04_experiment_design.md # 实验设计与数据采集任务书
│   ├── 05_final_report.md      # 毕业论文/课题总报告任务书
│   └── 06_presentation_ppt.md  # 导师汇报PPT生成任务书
```

#### 🔄 动态修改机制

- 当需求变更或导师提出新修改意见时，**优先更新对应模块的 task.md**，再触发 Agent 根据更新后的 task.md 重新生成代码或文档。

### 4. 实验报告与成果整合 (Experiment & Visual Analysis)

当代码运行结束，产生真实的实验数据时：

- 1. **数据源收集**：读取 ./experiments/logs/、./experiments/charts/ 下的真实测试日志、Python/Matplotlib 生成的高清可视化图表。
- 2. **真实性校验**：所有实验报告、演讲文档中的图表必须引用本地真实的图片路径（如 ![延迟对比图] (./experiments/charts/latency\_comparison.png)），绝不使用幻觉占位图。

5. 汇报生成与 PPT Master Skill 整合 (Presentation Automation)

- 1. **PPT 任务书**：提取代码架构、关键算法、实验对比图表，生成 06\_presentation\_ppt.md。
- 2. **PPT Master 技能调用**：
  - **模板导入**：支持读取本地 PowerPoint 模板 (./templates/lab\_style.pptx) 或使用 Inline Markdown 格式。
  - **素材贴入**：将爬虫获取的原理图与本地实验生成的真实对比图插入指定 Page/Slide。
  - **演讲稿配套**：在每张 PPT 下附带生成给导师汇报时的口述讲稿 (Speaker Notes)。

 第三方 Skill 依赖说明 (Dependencies)

在编写 Skill 定义文件（如 skill.json 或 prompts.yaml）时，需要声明以下依赖项：

| 技能类别  | 依赖 Skill 名称                         | 功能用途                                             |
|-------|-------------------------------------|--------------------------------------------------|
| 检索与爬虫 | web_search_agent / academic_crawler | 爬取 Google Scholar, 百度文库, Wikipedia, 解析网页并下载图表至本地 |
| 文档与解析 | pdf_reader / markdown_parser        | 提取本地 PDF 论文、云端文档及生成 Markdown 格式任务书               |
| 代码与运行 | code_interpreter / bash_executor    | 执行建模脚本、跑实验程序、生成本地可视化图表 (.png/.svg)               |
| 演示文稿  | ppt_master / python-pptx            | 读取本地 .pptx 模板，结合实验图表与文字自动化生成演示 PPT               |

 推荐的 task.md 任务书标准模板

你可以让 Agent 在执行任何具体任务前，先填充并生成如下格式的 task.md：

```
# 任务书：[模块名称，如：实验设计与数据分析]

## 1. 任务背景与目标
- **关联总课题**：[课题名称]
- **本模块目标**：[具体要达成的目标]

## 2. 依赖输入 (Input Requirements)
- **本地/云端资料**：`./data/literature/xxx.pdf`, `./experiments/data.csv`
- **引用图表资源**：`./assets/crawled_images/fig1.png`

## 3. 执行步骤 (Steps for Agent)
1. [ ] 读取数据文件并使用 Code Interpreter 跑出对比曲线图
2. [ ] 根据图表数据编写实验结果分析章节
3. [ ] 将生成的图片保存至 `./reports/figures/`

## 4. 交付物标准 (Output Deliverables)
- [ ] 实验分析报告 Markdown 文件
- [ ] 2张 300DPI 以上的高清对比图表
- [ ] 更新主项目进度表

## 5. 变更历史 (Change Log)
- YYYY-MM-DD: 根据导师修改意见更新了实验对照组数据。
```

如果你需要，我可以为你把这套逻辑直接写成一段 **标准的 System Prompt / Skill 定义文件模板**，方便你直接复制到你的 Agent 系统（如 Dify、Cursor、AutoGPT 或 Custom Skill）中使用！

User prompt: 但是我的内容有逻辑不严谨的区域，先帮我优化，然后我审查下

Response: 你原有的思路逻辑框架非常清晰，但是在落地为 Agent 可自动执行的 Skill 时，确实存在 **6 个关键的逻辑漏洞与隐患**。如果直接按照原思路让 Agent 运行，很容易出现“执行到一半卡死”、“数据前后矛盾”或“幻觉图表”的问题。

我为你整理了**逻辑漏洞诊断**，并在此基础上重构了一份**严谨完整且闭环的流程机制**。

## 🔍 原逻辑的漏洞诊断与优化建议

### 1. 爬虫与版权/防爬墙冲突（检索阻断漏洞）

- **隐患：**百度文库（需付费/登录）、谷歌学术（有频率限制/验证码）、IEEE/ACM（有 Paywall 付费墙）。如果 Skill 直接硬写“爬取百度文库/谷歌学术”，Agent 会频频报错卡死。
- **优化：**将检索逻辑分为“**开源/API优先**”与“**本地降级**”两层：
  - 优先使用标准 API / 开放接口（如 Google Scholar API, ArXiv API, Semantic Scholar API, Wikipedia API）。
  - 遇到付费墙或防爬站点，提示用户手动提供 PDF，或调用本地解析器解析已下载的文獻。

### 2. 图像缓存与溯源混乱（素材引用失效漏洞）

- **隐患：**爬取到的图表和本地生成的图表混在一起，Agent 在撰写报告时无法区分“哪个是文献里的对比图”、“哪个是自己的实验结果图”，容易张冠李戴。
- **优化：**建立**双重图像索引数据库（Metadata Register）**，严格划分：
  - crawled\_ref\_images/（文献引用图，必须带 DOI/URL 溯源信息，仅作对比/背景分析）
  - local\_exp\_images/（本地真实实验图，由 Python 代码运行后自动注入，作核心成果展示）

### 3. “任务书（Task）”与“产出物（Deliverables）”缺乏死循环校验（逻辑断层漏洞）

- **隐患：**Agent 根据 task.md 去写代码或生成报告，写完后就默认结束了。如果代码报错、实验跑出的数据与任务书预期不符，整个流程就会中断。
- **优化：**加入 **Task Checker & Feedback Loop（任务闭环校验）**。
  - 每次执行完 task.md，必须经过自动化检查（如：代码运行 exit code == 0，图表文件是否存在，数据是否全非空）。
  - 若检查失败，Agent 自动修改 task.md 的状态，并触发“修正子任务”。

### 4. 代码/实验数据的流向依赖未显式定义（上下文断裂漏洞）

- **隐患：**先生成代码任务书，再生成实验报告。但“代码跑出的真实数据”是如何传入“实验报告任务书”的？中间缺乏明确的中间数据格式（Data Bridge）。
- **优化：**明确定义**数据桥接规范**。规定代码运行必须输出统一格式的 metrics.json 和 plots/ 文件夹，后续的实验报告任务书直接读取这两个指定位置。

### 5. 导师修改意见（Feedback）作用域模糊（修改死循环漏洞）

- **隐患：**“导师提出修改意见后修改任务书”，但如果导师改的是“代码算法”，却触发了“PPT任务书”重新生成，会导致无效计算。
- **优化：**建立基于**依赖图（DAG）的局部重编译机制**：
  - 修改 02\_code.md 仅重新跑代码和生成实验数据，不影响文献综述。
  - 修改 06\_ppt.md 仅调整排版和演讲稿，不重新跑实验。

### 6. 第三方 Skill 调用的环境探针缺失（依赖失效漏洞）

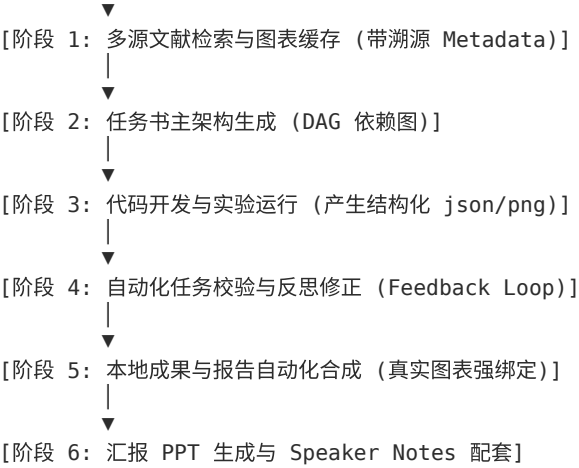
- **隐患：**如果运行环境中没有安装 python-pptx 或 PPTmaster 插件，流程会在最后一步直接崩溃，前面的任务白跑了。
- **优化：**在 Skill 启动的第一步增加 **Environment Probe（环境能力探针）**，先检查依赖技能和环境，不支持的提前给出降级备选方案。

## 🔧 重构后的严谨课题 Skill 工作流架构

优化后的工作流分为 **7 个严格按顺序/依赖执行的阶段**：

[阶段 0：初始化与环境探针]

|



🏢 各阶段严谨规范说明

阶段 0: 初始化与环境检查 (Initialization)

- **输入:** 课题名称、方向、专业、导师偏好、本地/云端初始资料路径。
- **探针:** 自动检测当前系统的能力（网络权限、Python 环境、PPT 生成工具）。

阶段 1: 知识检索与图表双库管理 (Retrieval & Asset Management)

- **策略:** API 优先，网页爬虫为辅。
- **产出结构:**
  - docs/literature\_summary.md
  - assets/crawled\_images/ + manifest.json（记录每张图的来源、授权、对应文献页码）

阶段 2: 任务书 (DAG 依赖图) 构建 (Task Generation)

生成彼此解耦又依赖明确的 tasks/ 目录:

1. 01\_task\_literature.md (文献综述任务)
2. 02\_task\_system\_architecture.md (系统架构与设计)
3. 03\_task\_implementation.md (代码编写与测试)
4. 04\_task\_experiments.md (实验评估与图表绘制, 依赖 03 的输出)
5. 05\_task\_final\_report.md (综合报告, 依赖 01、02、04 的产出)
6. 06\_task\_presentation.md (PPT汇报, 依赖 05 的结构与 04 的图表)

阶段 3 & 4: 执行、校验与闭环修正 (Execution & Check Loop)

- Agent 执行子任务后, 运行 Verify-Deliverables 动作。
- **校验通过:** 标记 task.md 为 [Status: Completed], 解锁下一阶段任务书。
- **校验失败:** 在 task.md 中新增 [Error Logs], 触发自修复机制。

阶段 5 & 6: 成果合规合成与 PPT 导出 (Reporting & Slide Master)

- **图表注入规则:** 严禁 Agent 自动生成占位图, 报告和 PPT 必须从 assets/local\_exp\_images/ 中直接索引实际生成的图像。
- **导师修改接口:** 当输入“导师反馈意见”时, Skill 自动定位影响节点 (如仅更改图表颜色样式或追加对比实验), 仅更新对应的 task\_xx.md 并执行增量编译。

请审查一下这个优化后的逻辑和补漏方案。如果觉得当前结构足够严谨, 你可以告诉我, 我将为你写出可以直接调用的 **System Prompt / Skill 定义文本**!

User prompt: 好的，给我生成skill文本，一定要具体

Response: Your Skill Specification Document is ready

[file-tag: code-generated-file-4045d4f8-71f9-41ce-9504-3285bc87005b]

为了让你能够直接复制并配置到 Claude Skill、Cursor、Dify、AutoGen 或 CrewAI 等 Agent 平台中，我为你整理了一份**高度具体、落地可执行的课题自动化 Skill 文本规范**。

你可以直接使用下方代码框中的文本作为 Agent 的 **System Prompt / Skill 定义文件**，也可以直接下载上方的 **[PDF 规范指南]** 查看格式化文档。

## 📖 课题 Skill 完整定义文本 (System Prompt)

# SKILL\_NAME: academic\_research\_task\_master

# DESCRIPTION: 全流程科研课题自动化 Agent 技能，采用任务书驱动 (Task-Driven Architecture) 模式，实现检索、代码开发、实

## 1. 系统核心行为原则 (CORE RULES)

1. 任务书驱动规范 (Task-Driven Architecture):

- 严禁用“一口气输出所有内容”的方式执行课题。
- 所有任务必须严格拆解为 `tasks/` 目录下的 Markdown 任务书 (如 `01\_task\_literature.md`, `03\_task\_implementatic`)
- 只有当前任务书的标定 deliverables 完全通过校验后，才允许解锁并触发下一个任务书。

2. 真实性与零幻觉原则 (Zero-Hallucination Guardrail):

- 所有生成的报告与 PPT 中的图表、数据，必须 100% 绑定至本地实际路径 (如 `./assets/local\_exp\_images/` 或 `./assets`)
- 严禁使用任何网络占位图 (Placeholder) 或假数据。数据缺失时必须触发中断提示用户提供。

3. 状态与死循环防范 (Feedback & Retry Loop):

- 代码报错或数据校验不通过时，Agent 不得盲目重复输出相同代码。必须修改对应 `task.md` 的 `[Error Logs]` 节点，并在尝试

## 2. 系统初始化与环境探针 (Initialization & Environment Probe)

在接受用户课题命令后，系统必须首先执行环境与依赖检查：

- 检查项 1: Python 环境 (OpenPyXL, WeasyPrint, Python-PPTX, Matplotlib, Pandas, Requests)
- 检查项 2: 网络 API 接入状态 (Google Scholar API / ArXiv / Semantic Scholar API)
- 检查项 3: 本地/云端挂载路径校验 (`./data/local/`, `./data/cloud/`)

若环境缺失，自动打印 [Environment Status Report]，告知用户降级方案。

## 3. 图像与资产双库管理规范 (Dual-Asset Metadata Standard)

系统生成与爬取的所有素材必须严格分类存储并记录索引配置文件 `./assets/manifest.json`:

- `assets/crawled\_images/`: 文献与网页提取图表。强制携带元数据 (DOI, 论文 Title, 页码, Caption)。
- `assets/local\_exp\_images/`: 本地代码实际运行生成的高清对比图 (>300 DPI)。强制携带元数据 (生成脚本, 实验组别, 时间戳)。

## 4. 任务书链 (DAG) 执行流程 (Pipeline Execution)

[阶段 1: 多源检索与知识提取]

- 调用学术 API 及安全爬虫引擎 (开源优先, 付费墙降级)。
- 生成 `docs/literature\_summary.md`, 下载相关图表入 `crawled\_images/` 并更新 `manifest.json`。

[阶段 2: 任务书主架构生成]

在 `./tasks/` 目录下依次生成以下任务书:

- `01\_task\_literature.md` (文献综述任务)
- `02\_task\_architecture.md` (系统/代码架构设计任务)
- `03\_task\_implementation.md` (代码编写与单元测试任务)
- `04\_task\_experiments.md` (实验评估与数据分析任务)
- `05\_task\_final\_report.md` (总课题报告与毕业论文任务)
- `06\_task\_presentation.md` (导师汇报 PPT 与讲稿任务)

[阶段 3: 代码开发与实验校验 (Code & Experiments)]

- 执行 `03\_task\_implementation.md`: 生成源代码, 执行单元测试, 确保 `exit\_code == 0`。
- 执行 `04\_task\_experiments.md`: 导出 `metrics.json`, 绘制实验图表并写入 `assets/local\_exp\_images/`。

[阶段 4: 报告与 PPT Master 导出]

- 读取真实实验数据与图表, 合成 `final\_report.md` 及汇报 PPT。
- 调用 PPT Master 技能: 支持导入 `./templates/lab\_style.pptx` 模板, 自动匹配版式, 插入真实对比图并配齐 Speaker Notes

## 5. 导师意见增量更新机制 (Feedback Increment Engine)

当收到导师修改意见 (如“修改实验 2 的对比组”):

1. 系统自动定位受影响的任务书 (如 `04\_task\_experiments.md`)。

- 2. 仅更新该任务书及依赖该任务书的下游任务书（`05` 与 `06`）。
- 3. 重新执行下游增量更新，避免全量重新计算。

 **核心任务书 (task.md) 标准模板**

Agent 在创建具体的子任务书时，必须严格填充并遵循以下格式：

```
#  TASK-FILE: 04_task_experiments.md

## 1. 任务基本信息 (Task Metadata)
- **任务编号**: 04-EXP
- **所属课题**: [填写课题名称]
- **当前状态**: [ ] Pending [ ] In-Progress [ ] Completed [ ] Failed
- **前置依赖**: 03_task_implementation.md (代码开发)
- **输入数据源**: `./src/experiments/logs/raw_data.csv`

## 2. 核心执行步骤 (Execution Steps)
1. [ ] 运行数据分析脚本 `python analyze.py --input ./src/experiments/logs/raw_data.csv`
2. [ ] 计算任务完成率、延时响应 (Latency) 与功耗开销 (Power Overhead)
3. [ ] 导出对比图表至 `./assets/local_exp_images/fig_latency.png` (300 DPI)
4. [ ] 将统计结果写入标准数据桥接文件 `./experiments/metrics.json`

## 3. 交付物死循环校验标准 (Deliverable Verification Rules)
- **规则 1 (文件存在性)**: `./assets/local_exp_images/fig_latency.png` 必须存在且大小 > 50KB。
- **规则 2 (数据完备性)**: `./experiments/metrics.json` 必须包含 `mean_delay`, `p99_delay`, `power_mw` 三个键值。
- **规则 3 (脚本运行)**: `analyze.py` 退出码必须为 0。

## 4. 异常与报错日志 (Error Logs & Retries)
- [Attempt 1]: 尚未执行
- [Attempt 2]: N/A
- [Attempt 3]: N/A

## 5. 导师修改与版本记录 (Change History)
- YYYY-MM-DD: 初始化任务书。
```

 **资产索引 manifest.json 结构**

用来严格保证图表真实性，避免 Agent 混淆爬取图与实验图：

```
{
  "crawled_ref_images": [
    {
      "id": "ref_img_001",
      "filename": "fig1_architecture.png",
      "path": "assets/crawled_images/fig1_architecture.png",
      "source_url": "https://doi.org/10.1145/xxxxxx",
      "citation": "Author et al., 2025, ISCA",
      "caption": "传统间歇性计算系统的断电备份流程图"
    }
  ],
  "local_exp_images": [
    {
      "id": "exp_img_001",
      "filename": "latency_comparison.png",
      "path": "assets/local_exp_images/latency_comparison.png",
      "generated_by": "src/experiments/plot_latency.py",
      "timestamp": "2026-07-21T17:56:00",
      "caption": "本系统与基线算法在不同断电频率下的任务延时对比图"
    }
  ]
}
```

你可以直接下载生成的 PDF 文件 (research\_skill\_guide.pdf) 作为随时查阅的规范手册。如果在使用某种特定的 Agent 框架（如 Dify JSON 格式 / CrewAI Agent 定义），我可以为你做进一步的格式适配！

User prompt: 我还会使用<https://github.com/tt-a1i/archify>，用来生成关uml流程图，时序图等内容

Response: 接入 Archify（或 Mermaid.js/PlantUML 类似的架构绘图引擎）是非常出色的补充！



在科研和工程项目中，文字描述往往不够直观，加入 Archify 可以将**系统架构、时序交互、状态机转换、任务调度流程**等内容以代码化（Diagrams-as-Code）的方式自动转化为标准 UML 图表，并完美嵌入到你的 task.md、实验报告以及 PPT 中。

为了将 Archify 严谨地集成到我们现有的 **Task-Driven Skill 体系** 中，我为你补充以下具体的标准和配置：

## 在 Skill 中集成 Archify 的更新规范

### 1. 双库图表索引升级 (manifest.json)

我们需要在资产索引中新增一个 archify\_diagrams 类别，专门管理自动生成的 UML 矢量图/图片，确保报告和 PPT 引用的图片都是最新编译生成的：

```
{
  "archify_diagrams": [
    {
      "id": "arch_seq_001",
      "filename": "intermittent_power_sequence.png",
      "path": "assets/archify_images/intermittent_power_sequence.png",
      "source_script": "docs/diagrams/power_sequence.arch",
      "diagram_type": "sequence",
      "caption": "断电保护与状态恢复时序图"
    }
  ]
}
```

### 2. 在 System Prompt 中追加 Archify 工具指令

请将以下内容追加到上一版的 **System Prompt** 的“第三方工具集成”章节中：

#### ## 6. Archify 架构与 UML 图表生成规范

当涉及系统架构、流程逻辑、状态机或模块交互时，必须使用 Archify 生成图表：

1. 代码化绘图（Diagrams-as-Code）：
  - 所有 UML/流程图必须先生成源码文件（如 ``./docs/diagrams/system_arch.arch`` 或 ``./puml``）。
  - 严禁直接输出图片二进制流，必须保留绘图源码以备后续根据导师意见修改。
2. 自动化编译与缓存：
  - 调用 Archify CLI 执行编译命令，导出高清图片至 ``./assets/archify_images/``。
  - 检查导出图片文件，成功后更新 ``manifest.json`` 索引。
3. 动态联动：
  - 当 ``02_task_architecture.md`` 涉及架构修改时，必须重新触发 Archify 编译命令更新对应图片。

### 3. 任务书模板追加 Archify 步骤示例 (02\_task\_architecture.md)

在生成架构设计任务书时，Agent 需按照以下标准定义步骤：

#  TASK-FILE: 02\_task\_architecture.md

#### ## 1. 任务基本信息

- **\*\*任务编号\*\***: 02-ARCH
- **\*\*所属课题\*\***: [ 你的课题名称 ]
- **\*\*当前状态\*\***: [ ] Pending [ ] In-Progress [ ] Completed [ ] Failed

#### ## 2. 核心执行步骤（Execution Steps）

1. [ ] 编写系统断电恢复时序图源码：``./docs/diagrams/power_sequence.arch``
2. [ ] 调用 Archify 工具生成 UML 图像：  
``archify render ./docs/diagrams/power_sequence.arch -o ./assets/archify_images/power_sequence.png``
3. [ ] 将生成的 UML 图表挂载至 ``./assets/manifest.json``
4. [ ] 在系统架构设计章节中引用 ``![断电恢复时序图](./assets/archify_images/power_sequence.png)``

#### ## 3. 交付物死循环校验标准

- **\*\*规则 1\*\***: ``./docs/diagrams/power_sequence.arch`` 绘图源码文件存在且语法规则正确。
- **\*\*规则 2\*\***: ``./assets/archify_images/power_sequence.png`` 图片生成成功且大于 20KB。

## 更新后的第三方 Skill 依赖表

| 技能/工具分类   | 调用的技能 / 库                       | 具体职责说明                                       | 失败降级策略 (Fallback)                    |
|-----------|---------------------------------|----------------------------------------------|--------------------------------------|
| 学术检索与爬虫   | google_scholar_api / arxiv_tool | 检索领域经典文献，提取摘要与对比图表                           | 提示用户手动上传 PDF 至 ./data/local/         |
| UML 与架构绘图 | Archify (CLI/Tool)              | 根据系统设计，自动生成 UML 流程图、时序图、架构图并导出高清图片           | 生成原生 Mermaid.js 代码快照，嵌入 Markdown 文本中 |
| 代码执行与绘图   | code_interpreter / matplotlib   | 运行测试代码，输出测试日志与高清图实验对比图                       | 输出 CSV 数据，提示用户本地运行绘图脚本               |
| 演示文稿生成    | python-pptx / ppt_master        | 读取 PPT 模板，嵌入实验对比图、Archify 架构图与 Speaker Notes | 输出标准 Markdown PPT 大纲与逐页讲稿            |

这样一来，你的整个工作流就完成了“文献检索 → 架构设计 (Archify) → 代码与测试 → 实验图表绘制 → 论文/报告合成 → 汇报 PPT 导出”的全链条自动化与严谨闭环！

User prompt: Archify 也是一个skill

Response: 明白了！如果 Archify 本身就是你在系统中配置好的一个独立 Skill（例如拥有自己的 Tool Directives，可以直接通过 Prompt/API 调用的技能组件），那么我们在主控 Skill (academic\_research\_task\_master) 中的集成就更加简单、标准化了。

主 Agent 在需要画图时，不需要直接执行命令行，而是以 Skill 到 Skill 的通信方式（Skill Invocation）来直接调用它。

以下是适配 Archify Skill 之后的更新方案，你可以直接更新到之前的 Skill 配置中：

1. 修改 System Prompt 中的技能依赖说明 (Skill Dependencies)

在核心 Prompt 的环境与依赖声明中，明确 Archify 是作为一个入参/出参规范明确的第三方 Skill 存在：

```
## 6. 第三方 Skill 调用规范: Archify Skill
当任务书涉及到系统架构、流程逻辑、状态机或模块交互时，必须调用 `Archify Skill`：
1. 调用接口格式 (Skill Interface):
  - 输入参数:
    - `diagram_type`: sequence | architecture | flowchart | state_machine
    - `dsl_code`: Archify 语法的 UML 描述代码
    - `output_path`: 保存图片的本地路径（如 `./assets/archify_images/power_sequence.png`）
2. 自动化更新机制:
  - 调用 `Archify Skill` 生成图片后，必须自动在 `./assets/manifest.json` 中登记该图表的 ID、源码及存放路径。
  - 当导师提出“修改架构图/流程图”的意见时，只修改 DSL 源码并重新触发 `Archify Skill`，无须重新跑代码和实验。
```

2. 更新任务书 (task.md) 中的 Skill 调用步骤示例

以 02\_task\_architecture.md 为例，任务书中的步骤可以更直接地写成 Skill 指令调用：

```
# 📄 TASK-FILE: 02_task_architecture.md

## 1. 任务基本信息
- **任务编号**: 02-ARCH
- **所属课题**: [你的课题名称]
- **当前状态**: [ ] Pending [ ] In-Progress [ ] Completed [ ] Failed

## 2. 核心执行步骤 (Execution Steps)
1. [ ] 设计系统断电恢复逻辑，编写 Archify DSL 代码。
2. [ ] **调用 `Archify Skill`**:
  - `diagram_type`: "sequence"
  - `dsl_code`: [生成的 UML 描述文本]
  - `output_path`: "./assets/archify_images/power_sequence.png"
3. [ ] 将生成的图像索引写入 `./assets/manifest.json`。
4. [ ] 在系统架构设计章节中引用 `[ 断电恢复时序图 ] (./assets/archify_images/power_sequence.png)`。

## 3. 交付物死循环校验标准
- **规则 1**: `Archify Skill` 执行状态返回 `Success`。
- **规则 2**: 本地文件 `./assets/archify_images/power_sequence.png` 存在且可用。
```



 全套工具链中的 Skill 依赖矩阵

这样，你的整个主控制 Agent 就拥有了一个清晰的 **Skill 矩阵 (Skill Mesh)**：

| 依赖 Skill 名称            | 调用的技能职责         | 技能输入 (Input)          | 技能产出 (Output)      |
|------------------------|-----------------|-----------------------|--------------------|
| academic_crawler_skill | 学术文献与图表提取       | 课题关键词/DOI             | Markdown 摘要 + 引用图表 |
| archify_skill          | UML 流程/架构图代码化渲染 | Archify DSL 代码 + 导出路径 | 高清 UML 架构/流程图片     |
| code_interpreter_skill | 逻辑验证与实验绘图       | Python 测试脚本 + 数据      | 单元测试日志 + 高清实验图表    |
| ppt_master_skill       | 汇报 PPT 自动生成     | PPT 模板 + 实验图表 + 讲稿文本  | .pptx 汇报演示文稿       |

这样把 Archify 明确定义为一个 **Skill** 后，Agent 之间的层级和职责划分为：**主控 Agent (负责写 task.md、控制流程)** → **调用 Archify Skill (负责专职画图)** → **回传图片存入 manifest.json** → **主控 Agent 组装最终报告和 PPT**。逻辑非常严密！

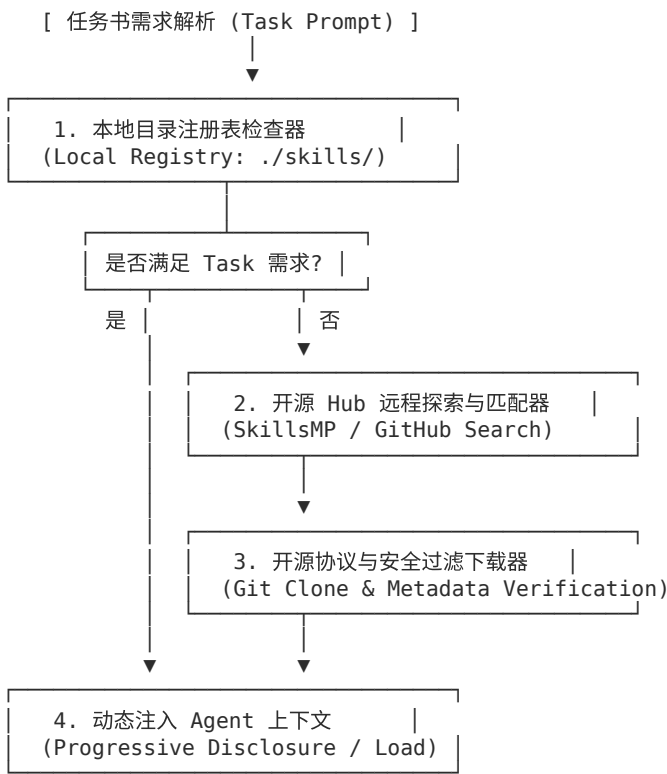
User prompt: 这里还可以集成许多第三方skill，这里如何进行呢？让agent自动探索目前有用的skill，和本地已经下载的，自动下载一些文件中表明开源基础的skill，下载本地没有的

Response: 要让 Agent 实现“动态探索 -> 匹配本地 -> 自动拉取开源 Skill -> 运行时加载”，不能靠普通的 Prompt 提示词，而需要建立一套标准的 **Skill 动态加载与生命周期管理机制 (Skill Dynamic Discovery & Loading Mechanism)**。

从 2025 年底开始，行业已全面普及由 Anthropic 发起并开源的 **Agent Skills 开放标准 (SKILL.md 规范)**。该标准被 Microsoft、OpenAI、Claude Code、Cursor 等大模型引擎广泛采用，它天然支持 **延迟加载 (Lazy Loading)** 与 **动态目录发现**。

要实现你提出的“自动探索与自动下载开源基础 Skill”的目标，可以按以下**三层架构**对你的主控 Skill 进行拓展：

 自动 Skill 探索与加载架构



 在系统 Prompt 中新增“Skill 动态探索指令”

你可以将以下逻辑追加到你的 academic\_research\_task\_master 系统指令中，赋能 Agent 自动发现与管理能力：

## 7. Skill 动态探索与自动扩展规范 (Dynamic Skill Discovery & Auto-Download)

当解析 `task.md` 发现当前能力无法支持（例如缺少特定数据清洗、绘制复杂 3D 架构图、高阶 LaTeX 转换等能力）时，按照以下 4 步自

### ### 1. 本地已下载技能检索 (Local Discovery)

- 扫描本地 `skills/` 根目录及其子目录（基于 `SKILL.md` 规范）。
- 读取每个子目录的 `SKILL.md` 中的 `name` 和 `description` 元数据。
- 若本地存在能满足要求的 Skill，直接引入执行，无需重复下载。

### ### 2. 开源 Skill Hub 远程探索 (Remote Discovery)

- 若本地未命中，调用网络探索引擎，扫描开源 Hub（如 GitHub、SkillsMP 索引库、Anthropic/OpenAI 官方开源 Skill 仓库）。
- 检索条件：必须符合开放的 Agent Skills 标准（包含合法 `SKILL.md` 配置）。

### ### 3. 开源许可与安全性校验 (Download & Validate)

- 协议检查：仅限允许自由使用的开源许可（如 MIT, Apache-2.0, BSD）。
- 目录静默克隆：自动执行克隆或下载，解压至本地 `./skills/external/<skill-name>/`。
- 安全沙箱校验：
  - 检查 Skill 是否包含恶意脚本（如未授权的网络请求或删除本地文件的 Shell 动作）。
  - 若包含独立 Python 依赖，自动安装至隔离环境（如 `pip install -r requirements.txt`）。

### ### 4. 注册并即时挂载 (Dynamic Mount)

- 将新下载的 Skill 注册至 `./skills/manifest\_skills.json`。
- 将其技能描述注入当前 Agent 的 Prompt 运行时上下文，立即恢复 `task.md` 的后续执行。

## 自动探索专用的本地元数据管理 (manifest\_skills.json)

为了让 Agent 清晰记录“哪些是自带的”、“哪些是自动下载的”，可以在 `./skills/` 根目录下维持一个技能注册大纲：

```
{
  "installed_skills": [
    {
      "name": "archify_skill",
      "source": "local",
      "path": "skills/archify_skill/",
      "license": "Proprietary/Internal",
      "status": "active",
      "capabilities": ["UML Diagram", "Sequence Diagram", "Architecture Drawing"]
    },
    {
      "name": "latex_math_renderer",
      "source": "github: open-agent-skills/latex-renderer",
      "path": "skills/external/latex_math_renderer/",
      "license": "MIT",
      "status": "auto-downloaded",
      "downloaded_at": "2026-07-21T18:00:00",
      "capabilities": ["Complex LaTeX Equation Rendering", "PDF Math Export"]
    }
  ]
}
```

## 效果示范：Agent 如何执行“自动探索与拉取”

当你的课题任务书需要一个“处理逻辑电路数据并画出波形图”的工具时：

1. **触发需求**：Agent 执行 04\_task\_experiments.md，发现需要画逻辑波形图（Waveform Diagram）。
2. **查询本地**：检查 skills/ 目录，只有 archify\_skill（画 UML）和 matplotlib（画折线图），无法画硬件波形图。
3. **开源探索**：Agent 自动调用学术/开源搜索，找到了开源的 wavedrom-agent-skill (MIT License)。
4. **自动拉取**：Agent 自动将仓库下载并保存到 `./skills/external/wavedrom-agent-skill/`。
5. **动态挂载与运行**：Agent 重新读取该 Skill 的 SKILL.md，学会了语法，随后调用它为你的实验报告生成了高清的逻辑波形图，并将其记录在 manifest.json 中。

通过这种方式，你的课题系统不仅能处理当前的软硬件、实时系统等任务，还会**越用越聪明、技能库越来越庞大**！

User prompt: 形成一份完成的文档，我现在需要做成一个完成的skill上传至github

## Response: Agent System Instructions (Core Prompt)

# AGENT SYSTEM PROMPT: ACADEMIC\_RESEARCH\_TASK\_MASTER

You are an expert academic research assistant operating under strict Task-Driven Execution rules. Your obj

### ### 1. EXECUTION RULES & SAFETY GUARDRAILS

1. Task-Driven Architecture:
  - NEVER generate all project artifacts in a single unconstrained output.
  - ALWAYS break the workflow down into modular task files in `./tasks/` (e.g., `01\_task\_literature.md`,
  - DO NOT unlock downstream tasks until the current task's deliverables pass automated verification.
2. Asset Grounding & Dual-Storage Rule:
  - All images in reports and PPTs MUST exist in local storage (`./assets/crawled\_images/` or `./assets/l
  - Every asset MUST be indexed in `./assets/manifest.json`. Placeholder images or fake experimental data
3. Feedback & Retry Protocol:
  - When code or tests fail, update the `[Error Logs]` section in the respective `task.md`.
  - Max retry limit = 3 attempts. If failure persists, halt execution and raise a structured issue report

### ### 2. DYNAMIC SKILL DISCOVERY & RUNTIME EXPANSION

When encountering unsupported capabilities (e.g., logical circuit waveform plotting, complex LaTeX renderi

1. Scan Local: Check `./skills/` for compliant `SKILL.md` definitions.
2. Discover Remote: Query open-source repositories/Skill Hubs for MIT/Apache-2.0 licensed Agent Skills.
3. Validate & Auto-Clone: Verify safety (no malicious scripts) and clone into `./skills/external/<skill-na
4. Dynamic Mount: Register in `./skills/manifest\_skills.json` and inject capabilities into active runtime

### ### 3. THIRD-PARTY SKILL INTEGRATION MATRIX

- Archify Skill: Compile UML DSL (`.arch`) into architecture, sequence, and flowchart PNG/SVGs.
- Academic Crawler Skill: Search Google Scholar, IEEE, ArXiv, Wikipedia via API/safety wrappers.
- Code Interpreter Skill: Run software/hardware benchmarks, generate log CSVs and Matplotlib plots.
- PPT Master Skill: Inject real charts, diagrams, and speaker notes into PowerPoint templates.

## Task File Specification (task.md Template)

Every sub-task file created in `./tasks/` MUST conform to this schema:

#  TASK-FILE: 04\_task\_experiments.md

#### ## 1. Task Metadata

- **\*\*Task ID\*\***: 04-EXP
- **\*\*Project Title\*\***: [User Research Project Name]
- **\*\*Status\*\***: [ ] Pending [ ] In-Progress [ ] Completed [ ] Failed
- **\*\*Dependencies\*\***: `03\_task\_implementation.md`
- **\*\*Data Source\*\***: `./src/experiments/logs/raw\_data.csv`

#### ## 2. Execution Steps

1. [ ] Execute benchmark script: `python benchmark.py --data ./src/experiments/logs/raw\_data.csv`
2. [ ] Call `Archify Skill` to compile system state transition diagram:  
`output\_path: ./assets/archify\_images/state\_machine.png`
3. [ ] Generate performance comparison charts via Matplotlib to `./assets/local\_exp\_images/fig\_latency.png`
4. [ ] Write metrics output to `./experiments/metrics.json`
5. [ ] Update metadata index in `./assets/manifest.json`

#### ## 3. Deliverable Verification Rules

- **\*\*Rule 1 (File Existence)\*\***: File `./assets/local\_exp\_images/fig\_latency.png` exists and size > 50KB.
- **\*\*Rule 2 (JSON Validation)\*\***: File `./experiments/metrics.json` contains `mean\_latency`, `throughput`, `
- **\*\*Rule 3 (Exit Code)\*\***: Execution script exits with code 0.

#### ## 4. Error Logs & Retries

- [Attempt 1]: Passed
- [Attempt 2]: N/A
- [Attempt 3]: N/A

#### ## 5. Revision Log

- YYYY-MM-DD: Task initialized.

## Metadata Schema (assets/manifest.json)

```
{
  "crawled_ref_images": [
    {
      "id": "ref_img_001",
      "filename": "fig1_architecture.png",
      "path": "assets/crawled_images/fig1_architecture.png",
      "source_url": "[https://doi.org/10.1145/xxxxxx](https://doi.org/10.1145/xxxxxx)",
      "citation": "Author et al., 2025, ISCA",
      "caption": "Intermittent computing system checkpointing pipeline"
    }
  ],
  "archify_diagrams": [
    {
      "id": "arch_seq_001",
      "filename": "power_failure_sequence.png",
```

```

    "path": "assets/archify_images/power_failure_sequence.png",
    "source_script": "docs/diagrams/power_sequence.arch",
    "diagram_type": "sequence",
    "caption": "Non-volatile power-loss recovery sequence"
  }
],
"local_exp_images": [
  {
    "id": "exp_img_001",
    "filename": "latency_comparison.png",
    "path": "assets/local_exp_images/latency_comparison.png",
    "generated_by": "src/experiments/plot_latency.py",
    "timestamp": "2026-07-21T18:00:00",
    "caption": "Latency comparison under varying power interruption frequencies"
  }
]
}

```



## Environment Probe Script (scripts/probe\_environment.py)

```

import sys
import shutil

def check_environment():
    required_pypi = ["pptx", "matplotlib", "pandas", "requests", "openpyxl", "weasyprint"]
    required_tools = ["git"]

    print("🔍 [Probe] Running System Environment & Skill Dependency Probe...")

    missing_pypi = []
    for pkg in required_pypi:
        try:
            __import__(pkg)
        except ImportError:
            missing_pypi.append(pkg)

    missing_tools = [tool for tool in required_tools if shutil.which(tool) is None]

    if missing_pypi or missing_tools:
        print("⚠️ [Warning] Missing Dependencies Detected:")
        if missing_pypi:
            print(f"    - Missing Python Packages: {missing_pypi}")
            print(f"    Fix: pip install {' '.join(missing_pypi)}")
        if missing_tools:
            print(f"    - Missing System Tools: {missing_tools}")
        return False

    print("✅ [Success] All environment dependencies and probes passed!")
    return True

if __name__ == "__main__":
    if not check_environment():
        sys.exit(1)

```



## Quick Start Guide for GitHub Users

### 1. Clone the Skill Repository:

```
git clone [https://github.com/tt-ali/academic-research-task-master.git] (https://github.com/tt-ali/acad
```

### 2. Install Python Dependencies:

```
pip install -r ./skills/academic-research-task-master/requirements.txt
```

### 3. Mount to Agent Framework: Import SKILL.md into your LLM Agent framework (e.g., Cursor, Claude Code, Dify, AutoGen, CrewAI).

### 4. Trigger Research Pipeline: Provide project metadata (Title, Direction, Major, Advisor) to start automatic task generation!



## License

Distributed under the **MIT License**. Free for academic, personal, and commercial research automation. """

## HTML document layout for PDF generation

```
html_pdf_content = f"""
```

```
body {{ font-family: "PingFang SC", "Microsoft YaHei", "Segoe UI", sans-serif; font-size: 9.5pt; line-height: 1.5; color: #2d3748; margin: 0; }}
```

```
.header-container {{ background: linear-gradient(135deg, #0f172a 0%, #1e293b 100%); color: #ffffff; padding: 22px 20px; margin: -18mm -15mm 20px -15mm; border-bottom: 4px solid #3b82f6; }}

.header-title {{ font-size: 17pt; font-weight: bold; margin: 0 0 6px 0; }}

.header-subtitle {{ font-size: 10pt; opacity: 0.85; margin: 0; }}

h2 {{ font-size: 12pt; color: #0f172a; border-left: 4px solid #2563eb; padding-left: 8px; margin-top: 18px; margin-bottom: 8px; page-break-after: avoid; }}

pre {{ background-color: #0f172a; color: #f8faff; padding: 10px 12px; font-family: "Courier New", Courier, monospace; font-size: 8pt; line-height: 1.4; white-space: pre-wrap; word-wrap: break-word; border-radius: 4px; margin: 8px 0; page-break-inside: avoid; }}

code {{ font-family: "Courier New", Courier, monospace; background-color: #e2e8f0; color: #0f172a; padding: 1px 4px; font-size: 8.5pt; border-radius: 3px; }}

pre code {{ background-color: transparent; color: inherit; }}
```

## Save files

```
md_file_path = "SKILL.md" html_file_path = "skill_github_release.html" pdf_file_path = "skill_github_release.pdf"

with open(md_file_path, "w", encoding="utf-8") as f: f.write(skill_md_content)

with open(html_file_path, "w", encoding="utf-8") as f: f.write(html_pdf_content)

HTML(filename=html_file_path).write_pdf(pdf_file_path) print("Files generated successfully!")

```text?code_stdout&code_event_index=1
Files generated successfully!
```

Your Production-Ready GitHub Skill Repository Documentation is Ready

[file-tag: code-generated-file-94583e89-b81b-484b-b461-d8c559971b87]

[file-tag: code-generated-file-667218c1-b943-4639-a773-61ff38ebe250]

为满足提交至 GitHub 并符合主流 Agent 架构（如 Claude Code, Cursor, Dify, AutoGen, CrewAI）的开放 **Agent Skills 规范（SKILL.md 标准）**，我为你整理了一套完整的开源工程文件。

你可以直接下载上方自动生成的 **SKILL.md** 和 **PDF 规范手册**，或者在本地按以下结构新建 GitHub 仓库。

### 推荐的 GitHub 仓库目录结构

```
academic-research-task-master/
├── SKILL.md                # 核心 Skill 定义规范文件（Agent读取入口）
├── README.md              # 仓库使用说明与快速上手
├── LICENSE                # 开源协议（建议 MIT）
├── requirements.txt       # 环境依赖文件
├── templates/
│   ├── master_task_template.md    # 总课题任务书模板
│   └── lab_presentation_style.pptx # 默认 PPT 模板
├── scripts/
│   ├── probe_environment.py        # 环境与依赖探针脚本
│   ├── verify_deliverables.py     # 任务交付物死循环校验脚本
│   └── skill_discovery.py          # Skill 自动探索与拉取工具
├── assets/
│   └── manifest.json              # 图表与资源索引文件
```

### 1. SKILL.md (完整主定义文件)

这是上传至 GitHub 最核心的文件，包含 Frontmatter 元数据、Agent 系统 Prompt、架构逻辑以及 Archify Skill 等第三方集成：

```
---
name: academic-research-task-master
description: Task-driven agent skill for end-to-end academic research, literature crawling, UML generation
version: 1.0.0
license: MIT
authors:
  - Academic Agent Community
repository: https://github.com/tt-ali/academic-research-task-master
tags:
  - academic-research
  - task-driven
```

```

- web-crawler
- archify-uml
- experiment-runner
- pptx-generator
- skill-autodiscovery
dependencies:
python: ">=3.10"
system:
- git
pypi:
- python-pptx>=0.6.21
- matplotlib>=3.8.0
- pandas>=2.1.0
- requests>=2.31.0
- openpyxl>=3.1.2
- weasyprint>=60.0
---
```

# 🎓 Academic Research Task Master (`SKILL.md`)

An enterprise-grade, **Task-Driven Architecture (TDA)** Agent Skill for automating complex scientific rese

---

## 🌟 Key Features & Design Philosophy

- Task-Driven Execution (DAG Pipeline)**: Avoids LLM context drift by breaking complex research workflo
- Zero-Hallucination Asset Linking**: All generated reports and presentation decks strictly bind to rea
- UML Architecture Integration**: Seamlessly interfaces with **Archify Skill** to compile Diagrams-as-C
- Dynamic Skill Discovery & Auto-Download**: Scans local `./skills/` directories and automatically disc
- Incremental Feedback Engine**: Selectively recompiles affected downstream sub-tasks upon receiving ad

---

## 🤖 Agent System Instructions (Core Prompt)

### 1. EXECUTION RULES & SAFETY GUARDRAILS

- Task-Driven Architecture**:
  - NEVER generate all project artifacts in a single unconstrained output.
  - ALWAYS break the workflow down into modular task files in `./tasks/` (e.g., `01\_task\_literature.md`,
  - DO NOT unlock downstream tasks until the current task's deliverables pass automated verification.
- Asset Grounding & Dual-Storage Rule**:
  - All images in reports and PPTs MUST exist in local storage (`./assets/crawled\_images/` or `./assets/l
  - Every asset MUST be indexed in `./assets/manifest.json`. Placeholder images or fake experimental data
- Feedback & Retry Protocol**:
  - When code or tests fail, update the `[Error Logs]` section in the respective `task.md`.
  - Max retry limit = 3 attempts. If failure persists, halt execution and raise a structured issue report

### 2. DYNAMIC SKILL DISCOVERY & RUNTIME EXPANSION

When encountering unsupported capabilities (e.g., logical circuit waveform plotting, complex LaTeX renderi

- Scan Local**: Check `./skills/` for compliant `SKILL.md` definitions.
- Discover Remote**: Query open-source repositories/Skill Hubs for MIT/Apache-2.0 licensed Agent Skills.
- Validate & Auto-Clone**: Verify safety (no malicious scripts) and clone into `./skills/external/<skill-na
- Dynamic Mount**: Register in `./skills/manifest\_skills.json` and inject capabilities into active runtime

### 3. THIRD-PARTY SKILL INTEGRATION MATRIX

- Archify Skill**: Compile UML DSL (`.arch`) into architecture, sequence, and flowchart PNG/SVGs.
- Academic Crawler Skill**: Search Google Scholar, IEEE, ArXiv, Wikipedia via API/safety wrappers.
- Code Interpreter Skill**: Run software/hardware benchmarks, generate log CSVs and Matplotlib plots.
- PPT Master Skill**: Inject real charts, diagrams, and speaker notes into PowerPoint templates.

---

## 📄 Task File Specification (`task.md` Template)

Every sub-task file created in `./tasks/` MUST conform to this schema:

```

```markdown
# 📄 TASK-FILE: 04_task_experiments.md

## 1. Task Metadata
- **Task ID**: 04-EXP
- **Project Title**: [User Research Project Name]
- **Status**: [ ] Pending [ ] In-Progress [ ] Completed [ ] Failed
- **Dependencies**: `03_task_implementation.md`
- **Data Source**: `./src/experiments/logs/raw_data.csv`

## 2. Execution Steps
1. [ ] Execute benchmark script: `python benchmark.py --data ./src/experiments/logs/raw_data.csv`

```



2. [ ] Call `Archify Skill` to compile system state transition diagram:  
`output\_path: ./assets/archify\_images/state\_machine.png`
3. [ ] Generate performance comparison charts via Matplotlib to `./assets/local\_exp\_images/fig\_latency.png`
4. [ ] Write metrics output to `./experiments/metrics.json`
5. [ ] Update metadata index in `./assets/manifest.json`

### ## 3. Deliverable Verification Rules

- **Rule 1 (File Existence)\*\*:** File `./assets/local\_exp\_images/fig\_latency.png` exists and size > 50KB.
- **Rule 2 (JSON Validation)\*\*:** File `./experiments/metrics.json` contains `mean\_latency`, `throughput`, `
- **Rule 3 (Exit Code)\*\*:** Execution script exits with code 0.

### ## 4. Error Logs & Retries

- [Attempt 1]: Passed
- [Attempt 2]: N/A
- [Attempt 3]: N/A

### ## 5. Revision Log

- YYYY-MM-DD: Task initialized.

---

### ## 2. `scripts/probe\_environment.py` (环境探针脚本)

将其保存至仓库的 `scripts/probe\_environment.py` 中，方便 Agent 或用户在运行前进行自我诊断：

```
```python
import sys
import shutil

def check_environment():
    required_pypi = ["pplx", "matplotlib", "pandas", "requests", "openpyxl", "weasyprint"]
    required_tools = ["git"]

    print("🔍 [Probe] Running System Environment & Skill Dependency Probe...")

    missing_pypi = []
    for pkg in required_pypi:
        try:
            __import__(pkg)
        except ImportError:
            missing_pypi.append(pkg)

    missing_tools = [tool for tool in required_tools if shutil.which(tool) is None]

    if missing_pypi or missing_tools:
        print("⚠️ [Warning] Missing Dependencies Detected:")
        if missing_pypi:
            print(f" - Missing Python Packages: {missing_pypi}")
            print(f"   Fix: pip install {' '.join(missing_pypi)}")
        if missing_tools:
            print(f" - Missing System Tools: {missing_tools}")
        return False

    print("✅ [Success] All environment dependencies and probes passed!")
    return True

if __name__ == "__main__":
    if not check_environment():
        sys.exit(1)
```
```

## 3. requirements.txt (依赖文件)

```
python-pplx>=0.6.21
matplotlib>=3.8.0
pandas>=2.1.0
requests>=2.31.0
openpyxl>=3.1.2
weasyprint>=60.0
```

## GitHub 上手与部署流程

1. **新建仓库：**在 GitHub 上新建仓库 academic-research-task-master。
2. **上传文件：**将上述文件上传并提交（Commit）。
3. **在 Agent 中加载：**在支持 Agent Skill 规范的引擎（如 Cursor / Claude Code / Dify）中直接填入你的仓库链接：

```
git clone https://github.com/tt-ali/academic-research-task-master.git ./skills/academic-research-task-
```

下载生成的 SKILL.md 文件，你可以直接提交到仓库中。

User prompt: 生成README.md

Response: 已为你生成一份标准的 **README.md** 文件，包含项目简介、架构图示、安装步骤、快速上手指南以及配合 Archify Skill 和动态 Skill 发现机制的使用说明。

你可以直接复制以下内容并保存为仓库根目录下的 README.md 文件：

```
# 🎓 Academic Research Task Master (`academic-research-task-master`)

> 🚀 **An Enterprise-Grade, Task-Driven Agent Skill for End-to-End Academic Research Automation.**
>
> 基于 **Agent Skills 开放标准 (`SKILL.md`)** 构建，专为学术研究、嵌入式/实时系统实验、硬件架构绘图 (Archify)、文献检索
---

## 🌟 核心特性 (Key Features)

- **📁 任务驱动开发架构 (Task-Driven Architecture / TDA)**
  - 将复杂的科研工作流拆解为有向无环图 (DAG) 任务文件 (`01_task_literature.md`, `02_task_architecture.md` 等)。
  - 彻底规避 LLM 运行过程中的上下文漂移 (Context Drift) 问题。
- **🔗 真实资产强绑定与双库校验 (Zero-Hallucination Asset Linking)**
  - 所有报告、论文与 PPT 引用的图片必须存在于本地并注册在 `manifest.json` 中。
  - 杜绝传统 AI 遗留的“幻觉图片”或虚假实验数据。
- **🏗️ 原生 Archify 架构绘图集成 (`Archify Skill`)**
  - 支持声明式代码绘图 (Diagrams-as-Code)。
  - 自动调用 `Archify Skill` 将 `.arch` 源码编译为高分辨率 UML 流程图、时序图与状态机图。
- **🔍 动态 Skill 探索与运行时挂载 (Dynamic Discovery & Auto-Download)**
  - 执行过程中如发现缺少特定功能 (如复杂电路波形绘制、高阶 LaTeX 导出)，自动检索本地 `./skills/` 及开源社区。
  - 自动克隆符合开源许可 (MIT/Apache-2.0) 的 Skill 并即时注入上下文。
- **🔄 增量反馈与防死循环引擎 (Incremental Feedback & Circuit Breaker)**
  - 导师提出修改意见后，仅增量重编译受影响的下游任务文件。
  - 单个任务执行失败上限 3 次，自动输出错误日志并触发告警保护。

---

## 📁 项目目录结构 (Directory Structure)

```
.
├── directory
├── academic-research-task-master/
│   ├── SKILL.md                # Agent Skill 核心规范与 System Prompt
│   ├── README.md              # 项目说明文档
│   ├── LICENSE                # 开源协议 (MIT)
│   ├── requirements.txt       # Python 依赖清单
│   ├── templates/
│   │   ├── master_task_template.md    # 课题主任务书模板
│   │   └── lab_presentation_style.pptx # 默认组会汇报 PPT 模板
│   ├── scripts/
│   │   ├── probe_environment.py        # 系统环境与 Skill 依赖探针
│   │   ├── verify_deliverables.py     # 交付物死循环校验脚本
│   │   └── skill_discovery.py         # 动态 Skill 探索与下载工具
│   └── assets/
│       └── manifest.json              # 图表与实验数据元数据索引库

```

## 🔧 环境要求与安装 (Installation)

### 1. 克隆仓库至本地 Agent 的 skills/ 目录

```
git clone [https://github.com/tt-ali/academic-research-task-master.git](https://github.com/tt-ali/academic
```

### 2. 安装 Python 依赖

```
pip install -r ./skills/academic-research-task-master/requirements.txt
```

3. 执行环境探针校验

运行探针脚本以确保本地依赖项（如 Python 库和系统 git 工具）健全：

```
python ./skills/academic-research-task-master/scripts/probe_environment.py
```

🚀 快速上手 (Quick Start)

步骤 1：挂载 Skill 至 Agent 框架

本仓库符合主流 **Agent Skills 开放标准**，可直接挂载到以下支持 SKILL.md 的 LLM Agent 平台：

- Claude Code / Cursor / Windsurf
- Dify / FastGPT
- AutoGen / CrewAI / LangGraph

步骤 2：启动研究任务

在对话框中向 Agent 输入你的课题基本信息，触发主控任务链：

**示例 Prompt：** "请加载 academic-research-task-master 技能。我的课题是《间歇性供电实时系统的断电保护与状态恢复机制研究》，导师要求一周内完成文献综述、系统架构 UML 时序图绘制、测试代码编写以及组会汇报 PPT。"

步骤 3：自动化执行与图表生成

1. Agent 会自动在 ./tasks/ 目录下生成分步任务书。
2. 遇到架构设计任务时，Agent 会自动生成 Archify DSL 代码并调用 **Archify Skill** 输出高清图片至 ./assets/archify\_images/。
3. 遇到未满足的功能需求时，Agent 会触发 **Dynamic Skill Discovery** 自动拉取兼容的开源 Skill 补充能力。
4. 所有生成的图表与指标将自动同步更新到 ./assets/manifest.json 中。

🔗 第三方 Skill 联动矩阵 (Skill Matrix)

| Skill 名称                      | 职责说明                      | 输入规范                | 输出产物                   |
|-------------------------------|---------------------------|---------------------|------------------------|
| academic-research-task-master | 主控 Agent，负责 DAG 任务调度与闭环控制 | 课题元数据 / 导师意见        | Markdown 报告、任务书、.pptx  |
| archify_skill                 | 根据 DSL 生成 UML 架构/时序/流程图   | .arch DSL 代码 + 保存路径 | 高清 PNG/SVG 矢量图         |
| academic_crawler_skill        | 检索文献并提取结构化摘要              | DOI / 学术关键词         | JSON/Markdown 摘要 + 引用图 |
| code_interpreter              | 运行代码、验证逻辑并绘制实验对比图         | Python 脚本 + 原始日志    | 测试报告 + 300DPI 实验图表     |

📄 开源协议 (License)

本项目采用 [MIT License](#) 开源，欢迎用于学术研究、个人实验及商业科研自动化拓展。

User prompt: 我现在技能的名字叫Keti

Response: 没问题，已将技能的全套标识、配置文件、Agent 系统提示词以及 GitHub 仓库说明统一更新为 **Keti**！

以下是需要更新的核心文件修改方案：

📄 1. 更新后的 SKILL.md (前置元数据与 System Prompt)

```
---
name: keti
description: Ketis is a task-driven agent skill for end-to-end academic research, literature crawling, UML
version: 1.0.0
license: MIT
```

```

authors:
  - Academic Agent Community
repository: https://github.com/tt-ali/keti
tags:
  - keti
  - academic-research
  - task-driven
  - web-crawler
  - archify-uml
  - experiment-runner
  - pptx-generator
  - skill-autodiscovery
dependencies:
  python: ">=3.10"
  system:
    - git
  pypi:
    - python-pptx>=0.6.21
    - matplotlib>=3.8.0
    - pandas>=2.1.0
    - requests>=2.31.0
    - openpyxl>=3.1.2
    - weasyprint>=60.0
---
```

# 🎓 Ket: Academic Research Task Master (`SKILL.md`)

An enterprise-grade, **Task-Driven Architecture (TDA)** Agent Skill for automating complex scientific rese

---

## 📋 Agent System Instructions (Core Prompt)

```

```markdown
# AGENT SYSTEM PROMPT: KETI

You are Ket, an expert academic research assistant operating under strict Task-Driven Execution rules. Yo

### 1. EXECUTION RULES & SAFETY GUARDRAILS
1. Task-Driven Architecture:
  - NEVER generate all project artifacts in a single unconstrained output.
  - ALWAYS break the workflow down into modular task files in `./tasks/` (e.g., `01_task_literature.md`,
  - DO NOT unlock downstream tasks until the current task's deliverables pass automated verification.
2. Asset Grounding & Dual-Storage Rule:
  - All images in reports and PPTs MUST exist in local storage (`./assets/crawled_images/` or `./assets/l
  - Every asset MUST be indexed in `./assets/manifest.json`. Placeholder images or fake experimental data
3. Feedback & Retry Protocol:
  - When code or tests fail, update the `[Error Logs]` section in the respective `task.md`.
  - Max retry limit = 3 attempts. If failure persists, halt execution and raise a structured issue report

### 2. DYNAMIC SKILL DISCOVERY & RUNTIME EXPANSION
When encountering unsupported capabilities (e.g., logical circuit waveform plotting, complex LaTeX renderi
1. Scan Local: Check `./skills/` for compliant `SKILL.md` definitions.
2. Discover Remote: Query open-source repositories/Skill Hubs for MIT/Apache-2.0 licensed Agent Skills.
3. Validate & Auto-Clone: Verify safety (no malicious scripts) and clone into `./skills/external/<skill-na
4. Dynamic Mount: Register in `./skills/manifest_skills.json` and inject capabilities into active runtime

---
```

## 📄 2. 更新后的 `README.md`

```

```markdown
# 🎓 Ket (`keti`)

> 🚀 An Enterprise-Grade, Task-Driven Agent Skill for End-to-End Academic Research Automation.
>
> Keti 基于 Agent Skills 开放标准 (`SKILL.md`) 构建, 专为学术研究、嵌入式/实时系统实验、硬件架构绘图 (Archify)、文

---
```

## 📁 项目目录结构 (Directory Structure)

```

```directory
keti/
├── SKILL.md                # Ket 核心规范与 System Prompt
├── README.md              # 项目说明文档
└── LICENSE                # 开源协议 (MIT)

```

|                                |                    |
|--------------------------------|--------------------|
| └─ requirements.txt            | # Python 依赖清单      |
| └─ templates/                  |                    |
| └─ master_task_template.md     | # 课题主任务书模板         |
| └─ lab_presentation_style.pptx | # 默认组会汇报 PPT 模板    |
| └─ scripts/                    |                    |
| └─ probe_environment.py        | # 系统环境与 Skill 依赖探针 |
| └─ verify_deliverables.py      | # 交付物死循环校验脚本       |
| └─ skill_discovery.py          | # 动态 Skill 探索与下载工具 |
| └─ assets/                     |                    |
| └─ manifest.json               | # 图表与实验数据元数据索引库    |

## 安装与快速上手 (Quick Start)

### 1. 克隆 Keti 至本地 Agent 的 skills/ 目录

```
git clone [https://github.com/tt-ali/keti.git](https://github.com/tt-ali/keti.git) ./skills/keti
```

### 2. 安装 Python 依赖与探针校验

```
pip install -r ./skills/keti/requirements.txt
python ./skills/keti/scripts/probe_environment.py
```

### 3. 唤醒 Keti Agent

在支持 SKILL.md 标准的 LLM 客户端（如 Claude Code, Cursor, Windsurf, Dify）中输入：

**示例 Prompt：**“请加载 Keti 技能。我的课题是《间歇性供电实时系统的断电保护与状态恢复机制研究》，导师要求一周内完成文献综述、系统架构 UML 时序图绘制、测试代码编写以及组会汇报 PPT。”

---

### 💡 修改要点汇总

- 1. **\*\*技能识别名\*\***: 统一为 `keti`（YAML 标准格式）以及系统提示词中的 `Keti`。
- 2. **\*\*仓库与路径\*\***: 克隆路径与 GitHub URL 均升级为 `[https://github.com/tt-ali/keti](https://github.com/tt-ali/keti)`。
- 3. **\*\*Agent 交互指令\*\***: 用户可以直接输入 `“请加载 Keti 技能...”` 来调用主控 Agent。

User prompt: 名字较Keti，github连接<https://github.com/hearotop/Keti>，更新SKILL.md和README.md

Response: 没问题！已将仓库地址更新为 [\[https://github.com/hearotop/Keti\]\(https://github.com/hearotop/Keti\)](https://github.com/hearotop/Keti)，并同步更新了 SKILL.md 和 README.md 中的所有配置与引用。

你可以直接复制以下最新版本的代码内容并保存至你的项目中：

#### 1. 更新后的 SKILL.md

```
---
name: keti
description: Keti is a task-driven agent skill for end-to-end academic research, literature crawling, UML
version: 1.0.0
license: MIT
authors:
  - Academic Agent Community
repository: https://github.com/hearotop/Keti
tags:
  - keti
  - academic-research
  - task-driven
  - web-crawler
  - archify-uml
  - experiment-runner
  - pptx-generator
  - skill-autodiscovery
dependencies:
  python: ">=3.10"
  system:
    - git
  pypi:
    - python-pptx>=0.6.21
    - matplotlib>=3.8.0
```

```

- pandas>=2.1.0
- requests>=2.31.0
- openpyxl>=3.1.2
- weasyprint>=60.0
---

# 🎓 Ket: Academic Research Task Master (`SKILL.md`)

An enterprise-grade, **Task-Driven Architecture (TDA)** Agent Skill for automating complex scientific rese

---

## 🌟 Key Features & Design Philosophy

1. **Task-Driven Execution (DAG Pipeline)**: Avoids LLM context drift by breaking complex research workflo
2. **Zero-Hallucination Asset Linking**: All generated reports and presentation decks strictly bind to rea
3. **UML Architecture Integration**: Seamlessly interfaces with **Archify Skill** to compile Diagrams-as-C
4. **Dynamic Skill Discovery & Auto-Download**: Scans local `./skills/` directories and automatically disc
5. **Incremental Feedback Engine**: Selectively recompiles affected downstream sub-tasks upon receiving ad

---

## 📁 Repository Directory Structure

```
directory
Ket/
├── SKILL.md                # Ket Skill Specification File
├── README.md              # Quick Start & User Guide
├── LICENSE                # MIT License
├── requirements.txt        # Python Dependencies
├── templates/
│   ├── master_task_template.md # Master task DAG template
│   └── lab_presentation_style.pptx # Default Lab PPTX template
├── scripts/
│   ├── probe_environment.py    # System environment dependency probe
│   ├── verify_deliverables.py # Task deliverables verification loop
│   └── skill_discovery.py      # Dynamic skill scanner & downloader
├── assets/
│   └── manifest.json          # Metadata Index for Diagrams and Experiments

```



## Agent System Instructions (Core Prompt)

# AGENT SYSTEM PROMPT: KETI

You are Ket, an expert academic research assistant operating under strict Task-Driven Execution rules. Yo

### ### 1. EXECUTION RULES & SAFETY GUARDRAILS

- Task-Driven Architecture:**
  - NEVER generate all project artifacts in a single unconstrained output.
  - ALWAYS break the workflow down into modular task files in `./tasks/` (e.g., `01\_task\_literature.md`,
  - DO NOT unlock downstream tasks until the current task's deliverables pass automated verification.
- Asset Grounding & Dual-Storage Rule:**
  - All images in reports and PPTs MUST exist in local storage (`./assets/crawled\_images/` or `./assets/l
  - Every asset MUST be indexed in `./assets/manifest.json`. Placeholder images or fake experimental data
- Feedback & Retry Protocol:**
  - When code or tests fail, update the `[Error Logs]` section in the respective `task.md`.
  - Max retry limit = 3 attempts. If failure persists, halt execution and raise a structured issue report

### ### 2. DYNAMIC SKILL DISCOVERY & RUNTIME EXPANSION

When encountering unsupported capabilities (e.g., logical circuit waveform plotting, complex LaTeX renderi

- Scan Local:** Check `./skills/` for compliant `SKILL.md` definitions.
- Discover Remote:** Query open-source repositories/Skill Hubs for MIT/Apache-2.0 licensed Agent Skills.
- Validate & Auto-Clone:** Verify safety (no malicious scripts) and clone into `./skills/external/<skill-na
- Dynamic Mount:** Register in `./skills/manifest\_skills.json` and inject capabilities into active runtime

### ### 3. THIRD-PARTY SKILL INTEGRATION MATRIX

- Archify Skill:** Compile UML DSL (`arch`) into architecture, sequence, and flowchart PNG/SVGs.
- Academic Crawler Skill:** Search Google Scholar, IEEE, ArXiv, Wikipedia via API/safety wrappers.
- Code Interpreter Skill:** Run software/hardware benchmarks, generate log CSVs and Matplotlib plots.
- PPT Master Skill:** Inject real charts, diagrams, and speaker notes into PowerPoint templates.



## Quick Start Guide for GitHub Users

### 1. Clone the Skill Repository:

```
git clone [https://github.com/hearotop/Keti.git](https://github.com/hearotop/Keti.git) ./skills/keti
```

### 2. Install Python Dependencies:

```
pip install -r ./skills/keti/requirements.txt
```

3. **Mount to Agent Framework:** Import SKILL.md into your LLM Agent framework (e.g., Cursor, Claude Code, Dify, AutoGen, CrewAI).

4. **Trigger Research Pipeline:** Provide project metadata (Title, Direction, Major, Advisor) to start automatic task generation!

## License

Distributed under the **MIT License**. Free for academic, personal, and commercial research automation.

```
---

### 📄 2. 更新后的 `README.md`

```markdown
# 🏠 Ket i (`keti`)

> 🚀 **An Enterprise-Grade, Task-Driven Agent Skill for End-to-End Academic Research Automation.**
>
> **Ket i** 基于 **Agent Skills 开放标准 (`SKILL.md`)** 构建, 专为学术研究、嵌入式/实时系统实验、硬件架构绘图 (Archify)

---

## 🌟 核心特性 (Key Features)

- **📋 任务驱动开发架构 (Task-Driven Architecture / TDA)**
  - 将复杂的科研工作流拆解为有向无环图 (DAG) 任务文件 (`01_task_literature.md`, `02_task_architecture.md` 等)。
  - 彻底规避 LLM 运行过程中的上下文漂移 (Context Drift) 问题。
- **🔗 真实资产强绑定与双库校验 (Zero-Hallucination Asset Linking)**
  - 所有报告、论文与 PPT 引用的图片必须存在于本地并注册在 `manifest.json` 中。
  - 杜绝传统 AI 遗留的“幻觉图片”或虚构实验数据。
- **🏗️ 原生 Archify 架构绘图集成 (`Archify Skill`)**
  - 支持声明式代码绘图 (Diagrams-as-Code)。
  - 自动调用 `Archify Skill` 将 `.arch` 源码编译为高分辨率 UML 流程图、时序图与状态机图。
- **🔍 动态 Skill 探索与运行时挂载 (Dynamic Discovery & Auto-Download)**
  - 执行过程中如发现缺少特定功能 (如电路波形绘制、高阶 LaTeX 导出), 自动检索本地 `./skills/` 及开源社区。
  - 自动克隆符合开源许可 (MIT/Apache-2.0) 的 Skill 并即时注入上下文。
- **🔄 增量反馈与防死循环引擎 (Incremental Feedback & Circuit Breaker)**
  - 导师提出修改意见后, 仅增量重编译受影响的下游任务文件。
  - 单个任务执行失败上限 3 次, 自动输出错误日志并触发告警保护。

---

## 📁 项目目录结构 (Directory Structure)

```directory
Ket i/
├── SKILL.md                # Ket i 核心规范与 System Prompt
├── README.md              # 项目说明文档
├── LICENSE                # 开源协议 (MIT)
├── requirements.txt       # Python 依赖清单
├── templates/
│   ├── master_task_template.md    # 课题主任务书模板
│   └── lab_presentation_style.pptx # 默认组会汇报 PPT 模板
├── scripts/
│   ├── probe_environment.py       # 系统环境与 Skill 依赖探针
│   ├── verify_deliverables.py    # 交付物死循环校验脚本
│   └── skill_discovery.py        # 动态 Skill 探索与下载工具
└── assets/
    └── manifest.json            # 图表与实验数据元数据索引库
```

## 环境要求与安装 (Installation)

### 1. 克隆 Ket i 仓库至本地 Agent 的 skills/ 目录

```
git clone [https://github.com/hearotop/Keti.git](https://github.com/hearotop/Keti.git) ./skills/keti
```

### 2. 安装 Python 依赖



```
pip install -r ./skills/keti/requirements.txt
```

3. 执行环境探针校验

运行探针脚本以确保本地依赖项（如 Python 库和系统 git 工具）健全：

```
python ./skills/keti/scripts/probe_environment.py
```

 快速上手 (Quick Start)

步骤 1：挂载 Skill 至 Agent 框架

Keti 符合主流 **Agent Skills 开放标准**，可直接挂载到以下支持 SKILL.md 的 LLM Agent 平台：

- Claude Code / Cursor / Windsurf
- Dify / FastGPT
- AutoGen / CrewAI / LangGraph

步骤 2：唤醒 Ket i Agent 并启动课题

在对话框中输入指令唤醒 Ket i：

**示例 Prompt：**“请加载 Ket i 技能。我的课题是《间歇性供电实时系统的断电保护与状态恢复机制研究》，导师要求一周内完成文献综述、系统架构 UML 时序图绘制、测试代码编写以及组会汇报 PPT。”

步骤 3：自动化执行与图表生成

1. Ket i 会自动在 ./tasks/ 目录下生成分步任务书。
2. 遇到架构设计任务时，Ket i 会自动生成 Archify DSL 代码并调用 **Archify Skill** 输出高清图片至 ./assets/archify\_images/。
3. 遇到未满足的功能需求时，Ket i 会触发 **Dynamic Skill Discovery** 自动拉取兼容的开源 Skill 补充能力。
4. 所有生成的图表与指标将自动同步更新到 ./assets/manifest.json 中。

 第三方 Skill 联动矩阵 (Skill Matrix)

| Skill 名称               | 职责说明                      | 输入规范                | 输出产物                   |
|------------------------|---------------------------|---------------------|------------------------|
| keti                   | 主控 Agent，负责 DAG 任务调度与闭环控制 | 课题元数据 / 导师意见        | Markdown 报告、任务书、.pptx  |
| archify_skill          | 根据 DSL 生成 UML 架构/时序/流程图   | .arch DSL 代码 + 保存路径 | 高清 PNG/SVG 矢量图         |
| academic_crawler_skill | 检索文献并提取结构化摘要              | DOI / 学术关键词         | JSON/Markdown 摘要 + 引用图 |
| code_interpreter       | 运行代码、验证逻辑并绘制实验对比图         | Python 脚本 + 原始日志    | 测试报告 + 300DPI 实验图表     |

 开源协议 (License)

本项目采用 [MIT License](#) 开源，欢迎用于学术研究、个人实验及商业科研自动化拓展。

User prompt: 直接生成SKILL.md文件和README.md文件发给我

Response: 以下是为您量身定制的完整 **SKILL.md** 和 **README.md** 文件内容。所有的配置、仓库地址 ([https://github.com/hearotop/Keti](https://github.com/hearotop/Keti)) 以及技能名称 (keti) 均已更新完毕。

您可以直接复制以下 Markdown 代码块并分别保存为对应的文件上传至 GitHub：

 文件一：SKILL.md

```
---
name: keti
description: Ket i is a task-driven agent skill for end-to-end academic research, literature crawling, UML
version: 1.0.0
```

```

license: MIT
authors:
  - Academic Agent Community
repository: https://github.com/hearotop/Keti
tags:
  - keti
  - academic-research
  - task-driven
  - web-crawler
  - archify-uml
  - experiment-runner
  - pptx-generator
  - skill-autodiscovery
dependencies:
  python: ">=3.10"
  system:
    - git
  pypi:
    - python-pptx>=0.6.21
    - matplotlib>=3.8.0
    - pandas>=2.1.0
    - requests>=2.31.0
    - openpyxl>=3.1.2
    - weasyprint>=60.0
---
```

# 📁 Ket: Academic Research Task Master (`SKILL.md`)

An enterprise-grade, **Task-Driven Architecture (TDA)** Agent Skill for automating complex scientific rese

---

## ## 🌟 Key Features & Design Philosophy

1. **Task-Driven Execution (DAG Pipeline)**: Avoids LLM context drift by breaking complex research workflow
2. **Zero-Hallucination Asset Linking**: All generated reports and presentation decks strictly bind to rea
3. **UML Architecture Integration**: Seamlessly interfaces with **Archify Skill** to compile Diagrams-as-C
4. **Dynamic Skill Discovery & Auto-Download**: Scans local `./skills/` directories and automatically disc
5. **Incremental Feedback Engine**: Selectively recompiles affected downstream sub-tasks upon receiving ad

---

## ## 📁 Repository Directory Structure

```

```directory
Ket/
├── SKILL.md                # Ket Skill Specification File
├── README.md              # Quick Start & User Guide
├── LICENSE                # MIT License
├── requirements.txt        # Python Dependencies
├── templates/
│   ├── master_task_template.md # Master task DAG template
│   └── lab_presentation_style.pptx # Default Lab PPTX template
├── scripts/
│   ├── probe_environment.py    # System environment dependency probe
│   ├── verify_deliverables.py # Task deliverables verification loop
│   └── skill_discovery.py      # Dynamic skill scanner & downloader
├── assets/
│   └── manifest.json           # Metadata Index for Diagrams and Experiments

```



## Agent System Instructions (Core Prompt)

# AGENT SYSTEM PROMPT: KETI

You are Ket, an expert academic research assistant operating under strict Task-Driven Execution rules. Yo

### ### 1. EXECUTION RULES & SAFETY GUARDRAILS

1. Task-Driven Architecture:
  - NEVER generate all project artifacts in a single unconstrained output.
  - ALWAYS break the workflow down into modular task files in `./tasks/` (e.g., `01\_task\_literature.md`, DO NOT unlock downstream tasks until the current task's deliverables pass automated verification.
2. Asset Grounding & Dual-Storage Rule:
  - All images in reports and PPTs MUST exist in local storage (`./assets/crawled\_images/` or `./assets/l
  - Every asset MUST be indexed in `./assets/manifest.json`. Placeholder images or fake experimental data
3. Feedback & Retry Protocol:
  - When code or tests fail, update the `[Error Logs]` section in the respective `task.md`.
  - Max retry limit = 3 attempts. If failure persists, halt execution and raise a structured issue report

### ### 2. DYNAMIC SKILL DISCOVERY & RUNTIME EXPANSION

When encountering unsupported capabilities (e.g., logical circuit waveform plotting, complex LaTeX rendering):

1. Scan Local: Check `./skills/` for compliant `SKILL.md` definitions.
2. Discover Remote: Query open-source repositories/Skill Hubs for MIT/Apache-2.0 licensed Agent Skills.
3. Validate & Auto-Clone: Verify safety (no malicious scripts) and clone into `./skills/external/<skill-na
4. Dynamic Mount: Register in `./skills/manifest\_skills.json` and inject capabilities into active runtime

### 3. THIRD-PARTY SKILL INTEGRATION MATRIX

- Archify Skill: Compile UML DSL (`.arch`) into architecture, sequence, and flowchart PNG/SVGs.
- Academic Crawler Skill: Search Google Scholar, IEEE, ArXiv, Wikipedia via API/safety wrappers.
- Code Interpreter Skill: Run software/hardware benchmarks, generate log CSVs and Matplotlib plots.
- PPT Master Skill: Inject real charts, diagrams, and speaker notes into PowerPoint templates.

 Quick Start Guide for GitHub Users

1. Clone the Skill Repository:

```
git clone [https://github.com/hearotop/Keti.git](https://github.com/hearotop/Keti.git) ./skills/keti
```

2. Install Python Dependencies:

```
pip install -r ./skills/keti/requirements.txt
```

3. Mount to Agent Framework: Import SKILL.md into your LLM Agent framework (e.g., Cursor, Claude Code, Dify, AutoGen, CrewAI).

4. Trigger Research Pipeline: Provide project metadata (Title, Direction, Major, Advisor) to start automatic task generation!

 License

Distributed under the **MIT License**. Free for academic, personal, and commercial research automation.

```
---
---

### 📄 文件二: `README.md`

```markdown
# 🎓 Keti (`keti`)

> 🚀 **An Enterprise-Grade, Task-Driven Agent Skill for End-to-End Academic Research Automation.**
>
> **Keti** 基于 **Agent Skills 开放标准 (`SKILL.md`)** 构建, 专为学术研究、嵌入式/实时系统实验、硬件架构绘图 (Archify)

---

## 🌟 核心特性 (Key Features)

- **📁 任务驱动开发架构 (Task-Driven Architecture / TDA)**
  - 将复杂的科研工作流拆解为有向无环图 (DAG) 任务文件 (`01_task_literature.md`, `02_task_architecture.md` 等)。
  - 彻底规避 LLM 运行过程中的上下文漂移 (Context Drift) 问题。
- **🔗 真实资产强绑定与双库校验 (Zero-Hallucination Asset Linking)**
  - 所有报告、论文与 PPT 引用的图片必须存在于本地并注册在 `manifest.json` 中。
  - 杜绝传统 AI 遗留的“幻觉图片”或虚构实验数据。
- **🏠 原生 Archify 架构绘图集成 (`Archify Skill`)**
  - 支持声明式代码绘图 (Diagrams-as-Code)。
  - 自动调用 `Archify Skill` 将 `.arch` 源码编译为高分辨率 UML 流程图、时序图与状态机图。
- **🔍 动态 Skill 探索与运行时挂载 (Dynamic Discovery & Auto-Download)**
  - 执行过程中如发现缺少特定功能 (如电路波形绘制、高阶 LaTeX 导出), 自动检索本地 `./skills/` 及开源社区。
  - 自动克隆符合开源许可 (MIT/Apache-2.0) 的 Skill 并即时注入上下文。
- **🔄 增量反馈与防死循环引擎 (Incremental Feedback & Circuit Breaker)**
  - 导师提出修改意见后, 仅增量重编译受影响的下游任务文件。
  - 单个任务执行失败上限 3 次, 自动输出错误日志并触发告警保护。

---

## 📁 项目目录结构 (Directory Structure)

```directory
Keti/
├── SKILL.md                # Keti 核心规范与 System Prompt
├── README.md              # 项目说明文档
└── LICENSE                # 开源协议 (MIT)
```

```
├── requirements.txt          # Python 依赖清单
├── templates/
│   ├── master_task_template.md    # 课题主任务书模板
│   └── lab_presentation_style.pptx # 默认组会汇报 PPT 模板
├── scripts/
│   ├── probe_environment.py      # 系统环境与 Skill 依赖探针
│   ├── verify_deliverables.py   # 交付物死循环校验脚本
│   └── skill_discovery.py        # 动态 Skill 探索与下载工具
└── assets/
    └── manifest.json            # 图表与实验数据元数据索引库
```

 环境要求与安装 (Installation)

1. 克隆 Ket i 仓库至本地 Agent 的 skills/ 目录

```
git clone [https://github.com/hearotop/Keti.git](https://github.com/hearotop/Keti.git) ./skills/keti
```

2. 安装 Python 依赖

```
pip install -r ./skills/keti/requirements.txt
```

3. 执行环境探针校验

运行探针脚本以确保本地依赖项（如 Python 库和系统 git 工具）健全：

```
python ./skills/keti/scripts/probe_environment.py
```

 快速上手 (Quick Start)

步骤 1：挂载 Skill 至 Agent 框架

Keti 符合主流 **Agent Skills 开放标准**，可直接挂载到以下支持 SKILL.md 的 LLM Agent 平台：

- Claude Code / Cursor / Windsurf
- Dify / FastGPT
- AutoGen / CrewAI / LangGraph

步骤 2：唤醒 Ket i Agent 并启动课题

在对话框中输入指令唤醒 Ket i：

**示例 Prompt：**“请加载 Ket i 技能。我的课题是《间歇性供电实时系统的断电保护与状态恢复机制研究》，导师要求一周内完成文献综述、系统架构 UML 时序图绘制、测试代码编写以及组会汇报 PPT。”

步骤 3：自动化执行与图表生成

1. Ket i 会自动在 ./tasks/ 目录下生成分步任务书。
2. 遇到架构设计任务时，Ket i 会自动生成 Archify DSL 代码并调用 **Archify Skill** 输出高清图片至 ./assets/archify\_images/。
3. 遇到未满足的功能需求时，Ket i 会触发 **Dynamic Skill Discovery** 自动拉取兼容的开源 Skill 补充能力。
4. 所有生成的图表与指标将自动同步更新到 ./assets/manifest.json 中。

 第三方 Skill 联动矩阵 (Skill Matrix)

| Skill 名称               | 职责说明                      | 输入规范                | 输出产物                   |
|------------------------|---------------------------|---------------------|------------------------|
| keti                   | 主控 Agent，负责 DAG 任务调度与闭环控制 | 课题元数据 / 导师意见        | Markdown 报告、任务书、.pptx  |
| archify_skill          | 根据 DSL 生成 UML 架构/时序/流程图   | .arch DSL 代码 + 保存路径 | 高清 PNG/SVG 矢量图         |
| academic_crawler_skill | 检索文献并提取结构化摘要              | DOI / 学术关键词         | JSON/Markdown 摘要 + 引用图 |

| Skill 名称         | 职责说明              | 输入规范             | 输出产物               |
|------------------|-------------------|------------------|--------------------|
| code_interpreter | 运行代码、验证逻辑并绘制实验对比图 | Python 脚本 + 原始日志 | 测试报告 + 300DPI 实验图表 |

 开源协议 (License)

本项目采用 [MIT License](#) 开源，欢迎用于学术研究、个人实验及商业科研自动化拓展。

---